

Мета роботи

Набуття практичних навичок моделювання програмних систем із використанням уніфікованої мови моделювання UML (Unified Modeling Language). Оволодіння методами побудови діаграм прецедентів (Use Case Diagram), діаграм класів (Class Diagram) та діаграм послідовності (Sequence Diagram). Формування вміння забезпечувати трасовність (traceability) від функціональних вимог до моделей проектування.

У результаті виконання роботи здобувач має продемонструвати здатність: обирати предметну область та формулювати функціональні вимоги; застосовувати нотацію UML 2.5 для візуального моделювання архітектури програмної системи; будувати матрицю трасовності, що пов’язує вимоги, прецеденти, класи та сценарії взаємодії.

FaceCloude

ID вимоги	Функціональна вимога	Прецедент (Use Case)	Класи
FR-01	Реєстрація та авторизація користувачів	Реєстрація, Авторизація	User
FR-02	Створення, редагування та видалення публікацій	Створення публікації	User, Post
FR-03	Перегляд стрічки новин	Перегляд стрічки новин	User, Post
FR-04	Кастомізація профілю користувача	Редагування профілю	User, Profile
FR-05	Система сповіщень про нові події	Отримання сповіщень	Notification, User
FR-06	Обмін приватними повідомленнями	Приватні повідомлення	User, Message
FR-07	Блокування користувачів адміністратором	Блокування користувача	Admin, User
FR-08	Видалення неприйнятного контенту адміністратором	Видалення контенту	Admin, Post

Висновок до матриці трасовності

Матриця трасовності демонструє зв'язок між функціональними вимогами, прецедентами використання, класами системи. Аналіз матриці показує, що всі функціональні вимоги мають відповідне покриття елементами проектування, що забезпечує повноту та узгодженість моделі системи FaceCloude. Це дозволяє простежити реалізацію кожної вимоги на різних етапах розробки програмного забезпечення.

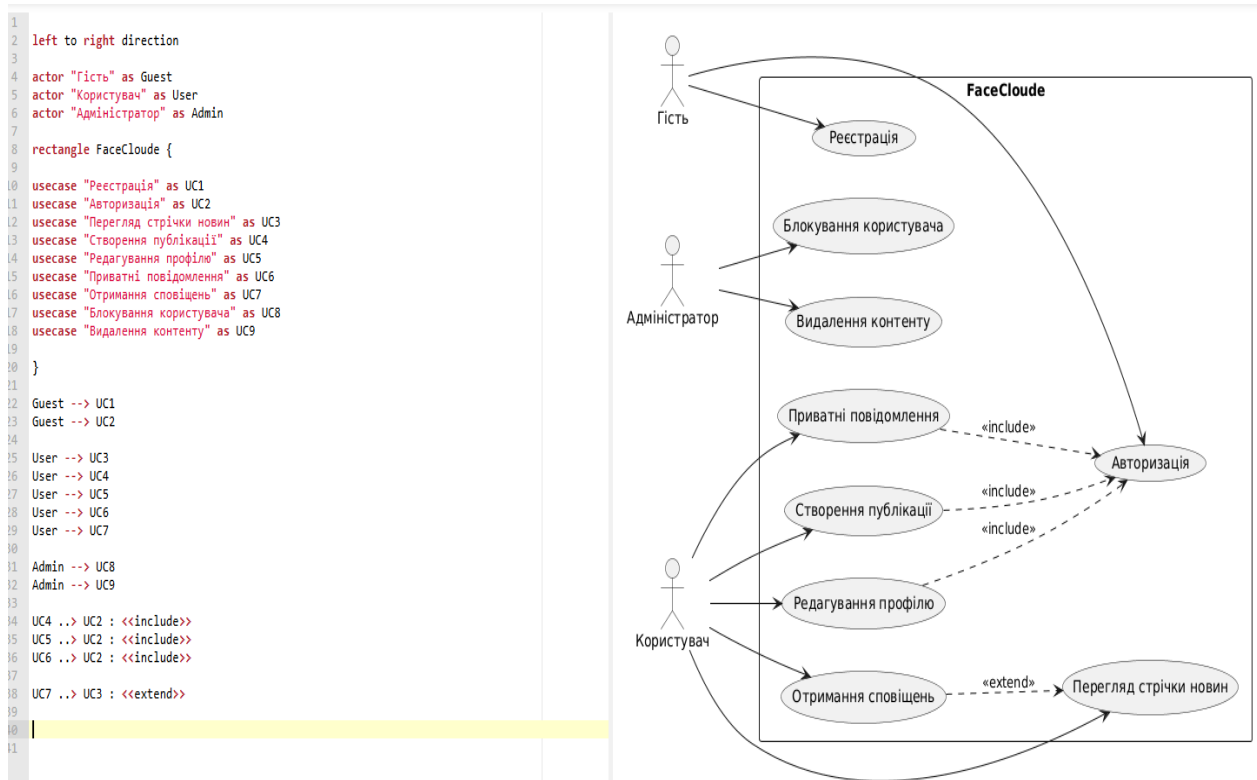
FaceCloud — це веборієнтована соціальна мережа, призначена для спілкування користувачів, обміну контентом та взаємодії між учасниками платформи. Система надає можливість створювати та переглядати публікації, налаштовувати власний профіль, обмінюватися приватними повідомленнями та отримувати сповіщення про нові події.

Основною метою проєкту є створення зручного середовища для спілкування користувачів різних професій та інтересів. Система підтримує ролі гостя, зареєстрованого користувача та адміністратора. Адміністратор має розширені права для модерації контенту та керування обліковими записами користувачів.

Основні функції системи:

- реєстрація та авторизація користувачів;
- перегляд стрічки новин;
- створення, редагування та видалення публікацій;
- налаштування профілю користувача;
- обмін приватними повідомленнями;
- система сповіщень;
- адміністрування та модерація контенту.

Проєкт розробляється як сучасний вебсервіс із підтримкою мобільних та настільних браузерів і забезпеченням захисту персональних даних користувачів.



@startuml

left to right direction

actor "Гість" as Guest

actor "Користувач" as User

actor "Адміністратор" as Admin

rectangle FaceCloude {

usecase "Реєстрація" as UC1

usecase "Авторизація" as UC2

usecase "Перегляд стрічки новин" as UC3

usecase "Створення публікації" as UC4

usecase "Редагування профілю" as UC5

usecase "Приватні повідомлення" as UC6

usecase "Отримання сповіщень" as UC7

usecase "Блокування користувача" as UC8

usecase "Видалення контенту" as UC9

}

Guest --> UC1

Guest --> UC2

User --> UC3

User --> UC4

User --> UC5

User --> UC6

User --> UC7

Admin --> UC8

Admin --> UC9

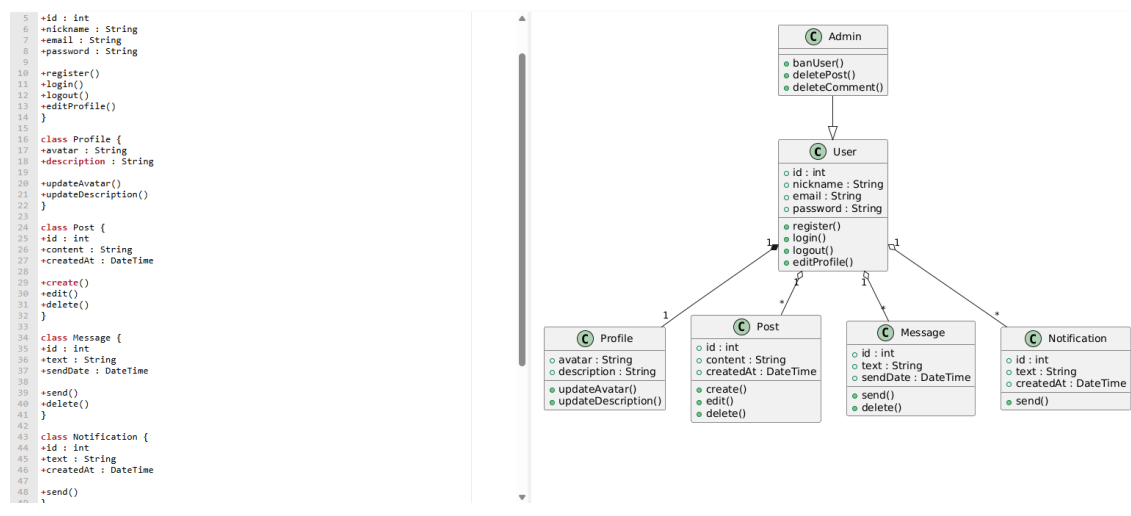
UC4 ..> UC2 : <<include>>

UC5 ..> UC2 : <<include>>

UC6 ..> UC2 : <<include>>

UC7 ..> UC3 : <<extend>>

@enduml



@startuml

```
class User {  
  
+id : int  
  
+nickname : String  
  
+email : String  
  
+password : String  
  
  
+register()  
  
+login()  
  
+logout()  
  
+editProfile()  
  
}
```

```
class Profile {  
+avatar : String  
+description : String  
  
+updateAvatar()  
+updateDescription()  
}
```

```
class Post {  
+id : int  
+content : String  
+createdAt : DateTime  
  
+create()  
+edit()  
+delete()  
}
```

```
class Message {  
+id : int  
+text : String  
+sendDate : DateTime  
  
+send()  
+delete()  
}
```

```
class Notification {  
+id : int  
+text : String  
+createdAt : DateTime
```

```
+send()
}
```

```
class Admin {
+banUser()
+deletePost()
+deleteComment()
}
```

Admin --|> User

User "1" *-- "1" Profile

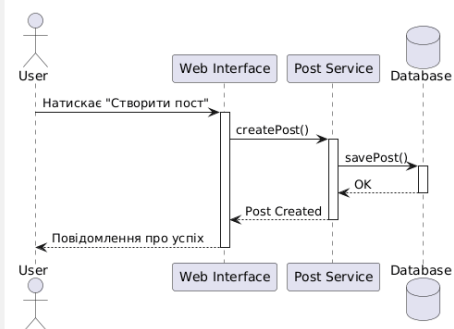
User "1" o-- "*" Post

User "1" o-- "*" Message

User "1" o-- "*" Notification

@enduml

```
1 @startuml
2 actor User
3 participant "Web Interface" as UI
4 participant "Post Service" as Service
5 participant "Database" as DB
6
7 User -> UI : Натискає "Створити пост"
8
9 activate UI
10
11 UI -> Service : createPost()
12
13 activate Service
14
15 Service -> DB : savePost()
16
17 activate DB
18
19 DB --> Service : OK
20
21 deactivate DB
22
23 Service --> UI : Post Created
24
25 deactivate Service
26
27 UI --> User : Повідомлення про успіх
28
29 deactivate UI
30
31 @enduml
```



@startuml

actor User

participant "Web Interface" as UI

participant "Post Service" as Service

database "Database" as DB

User -> UI : Натискає "Створити пост"

activate UI

UI -> Service : createPost()

activate Service

Service -> DB : savePost()

activate DB

DB --> Service : OK

deactivate DB

Service --> UI : Post Created

deactivate Service

UI --> User : Повідомлення про успіх

deactivate UI

@enduml